

KATEGORYZATOR

System Automatycznej Kategoryzacji Dokumentów Księgowych

Raport Końcowy Projektu

Projekt: Kategoryzator – ML-powered kategoryzacja faktur

Wersja: 2.0.0

Data: Maj 2026

Technologie: Python · TensorFlow/Keras · XGBoost · FastAPI · AWS Bedrock · Docker

Profile: KPIR (Księga Przychodów i Rozchodów) + ADVANCED (Ryczałt)

Autorzy: Łukasz Byrka | Paweł Włodarczyk

Autorzy

Łukasz Byrka | Paweł Włodarczyk

Spis Treści

1. Streszczenie wykonawcze
2. Cel i kontekst biznesowy projektu
3. Architektura systemu
4. Dane treningowe
5. Metodologia i Feature Engineering
6. Modele uczenia maszynowego
 - 6.1 Keras – sieci neuronowe
 - 6.2 XGBoost – gradient boosting
 - 6.3 Hyperparameter Tuning
7. Wyniki i metryki modeli
 - 7.1 Wyniki XGBoost (KPIR)
 - 7.2 Wyniki Keras (KPIR)
 - 7.3 Wyniki ADVANCED (Ryczałt)
8. Wykresy z trenowania
9. Architektura API
 - 9.1 Endpointy REST
 - 9.2 Flow decyzyjny
 - 9.3 Fallback LLM (AWS Bedrock)
 - 9.4 Historia księgowości (SQLite)
10. Infrastruktura i wdrożenie
11. Testy integracyjne
12. Przypadki testowe (UAT – wyniki rzeczywiste)
13. Wnioski końcowe
14. Rekomendacje i dalszy rozwój

1. Streszczenie wykonawcze

Kategoryzator to system automatycznej klasyfikacji dokumentów księgowych, zbudowany na potrzeby platformy Symfonia. System wykorzystuje techniki uczenia maszynowego (sieci neuronowe Keras oraz gradient boosting XGBoost) do przypisywania faktur kosztowych i przychodowych do właściwych kategorii podatkowych. Zadaniem systemu jest wspomaganie i automatyzacja pracy księgowego poprzez propozycje kategoryzacji dla każdego nowego dokumentu.

Projekt dostarcza kompletne rozwiązanie produkcyjne obejmujące: trening i ewaluację modeli ML, REST API (FastAPI) z logiką wielostopniowej decyzji, integrację z historią księgową (SQLite + FTS5), mechanizm fallback do dużych modeli językowych (LLM) przez AWS Bedrock, konteneryzację Docker oraz konsumenta kolejki SQS.

Kluczowe wyniki

Obszar	Wynik
Accuracy XGBoost:	od 79.7% (metoda_rozliczenia_vat) do 99.8% (srodek_trwaly)
Accuracy Keras:	od ~80% (metoda_rozliczenia_podatku) do ~99.8% (srodek_trwaly)
Ensemble:	Podwójne potwierdzenie (XGBoost + Keras); rozbieżność kieruje do weryfikacji
LLM Fallback:	AWS Bedrock przy niskiej pewności historycznej i gdy firma włączyła LLM
Wdrożenie:	Docker, AWS EC2/ECS, opcjonalnie SQS consumer

2. Cel i kontekst biznesowy projektu

Każda firma prowadząca działalność gospodarczą w Polsce jest zobowiązana do prawidłowego klasyfikowania dokumentów księgowych – faktur zakupu i sprzedaży – zgodnie z właściwym reżimem podatkowym. W praktyce oznacza to konieczność przypisania do każdego dokumentu kilku atrybutów jednocześnie:

- kolumna_kpir – przynależność do właściwej kolumny Księgi Przychodów i Rozchodów
- metoda_rozliczenia_podatku – sposób odliczenia kosztu (STD100, STD75, STD20, STD0 ...)
- metoda_rozliczenia_vat – metoda rozliczenia podatku VAT (VAT100, VAT50, VAT50NETTO ...)
- odliczenie_vat – szczegółowe odliczenie VAT zgodnie z art. ustawy o VAT
- cel_zakupu – cel dla którego zakup jest dokonany (TAXED_SALE, EXEMPT_SALE ...)
- srodek_trwaly – czy dokument dotyczy środka trwałego (PRAWDA / FALSE)

Ręczna klasyfikacja jest czasochłonna, podatna na błędy i wymaga wiedzy specjalistycznej. Kategorizator rozwiązuje ten problem poprzez automatyczne przewidywanie kategorii na podstawie: nazwy dokumentu, typu pozycji (EXPENDITURE / INCOME) oraz identyfikatora firmy – personalizacja per klient. Biznesowe korzyści to skrócenie czasu księgowania, redukcja błędów klasyfikacji, pełna audytowalność decyzji AI oraz możliwość doskonalenia systemu przez mechanizm feedbacku (endpoint /bookkeeping/confirm).

3. Architektura systemu

Warstwa	Opis
Warstwa danych treningowych	Pliki CSV (dane_ai.csv – KPIR, dane_ai_ryczalt.csv – ADVANCED) zawierające historyczne dokumenty z poprawnymi kategoriami.
Warstwa ML (trening)	Skrypty Python w katalogu training/ realizujące pełny pipeline: czyszczenie danych, feature engineering, trening modeli Keras i XGBoost, zapis modeli i enkoderów.
Warstwa predykcji (inference)	Moduły api/services/ ładujące wytrenowane artefakty i realizujące predykcję online.
Warstwa API	FastAPI (api/app.py) – REST API z endpointami do predykcji, zarządzania historią, konfiguracji per firma i raportowania LLM.
Warstwa historii	SQLite z FTS5 – przechowuje potwierdzone przez księgowego kategoryzacje; używana do exact-match lookup przed uruchomieniem modeli ML.
Warstwa LLM (fallback)	AWS Bedrock (Converse API) – wywołany gdy pewność historyczna jest zbyt niska i firma wyraziła zgodę na użycie LLM.
Infrastruktura	Docker + AWS EC2/ECS; opcjonalny SQS consumer do asynchronicznego przetwarzania.

3.1 Flow decyzyjny predykcji

Krok	Opis	Status
Krok 1	API przyjmuje request: nazwa, typ_pozycji, company_id, accounting_type	—
Krok 2	Pobrana jest konfiguracja firmy (progi pewności, flaga llm_enabled)	—
Krok 3	Szukanie podobnej faktury w historii SQLite (FTS5 + cosine similarity)	—
Krok 4	similarity $\geq$ confidence_exact $\rightarrow$ zwrot historycznego wyniku	historical_match
Krok 5	Brak dopasowania $\rightarrow$ uruchomienie XGBoost i Keras równolegle	—
Krok 6	Modele zgodne $\rightarrow$ zwrot predykcji	model_consensus
Krok 7	Modele rozbieżne $\rightarrow$ obie podpowiedzi do wyboru	manual_review_required
Krok 8	similarity < confidence_ai $\rightarrow$ opcjonalny LLM fallback	ai_assist_required
Krok 9	Księgowy potwierdza $\rightarrow$ zapis do historii SQLite	saved

4. Dane treningowe

4.1 Plik dane_ai.csv (profil KPIR)

Kolumna	Opis
document_id	Unikalny identyfikator dokumentu
nazwa	Oryginalna nazwa dokumentu/faktury – główna cecha wejściowa
typ_pozycji	Typ dokumentu: EXPENDITURE (koszt) lub INCOME (przychód)
company_id	Identyfikator firmy – personalizacja modelu
typ	Typ transakcji; filtr: BOOK (tylko rzeczywiste księgowania)
kolumna_kpir	Cel 1: kolumna KPIR (OTHER_EXPENSES, PURCHASE_OF_TRADING_GOODS, ...)
metoda_rozliczenia_podatku	Cel 2: metoda podatkowa (STD100, STD75, STD20, STD0, ...)
metoda_rozliczenia_vat	Cel 3: metoda VAT (VAT100, VAT50, VAT50NETTO, VAT0, ...)
odliczenie_vat	Cel 4: szczegółowe odliczenie VAT (DEDUCTION_VAT_S_*)
cel_zakupu	Cel 5: cel zakupu (TAXED_SALE, EXEMPT_SALE, TAXED_AND_EXEMPT_SALE)
srodek_trwaly	Cel 6: środek trwały (PRAWDA / FALSE)

4.2 Plik dane_ai_ryczałt.csv (profil ADVANCED)

Dane dla reżimu ryczałtowego zawierają te same cechy wejściowe, lecz bez kolumny kolumna_kpir (nieaplikowalne dla ryczałtu). Uwaga: w danych występuje literówka metoda_rozliczenia_podatku (zamiast metoda_...) – obsługiwana przez alias w kodzie.

4.3 Podział Train/Test

Dane dzielone są w stosunku 70%/30% (train/test) ze stałym ziarnem losowości (random_state=42) w celu zapewnienia reprodukowalności wyników. Feature engineering (TF-IDF, OneHotEncoder) fitowany jest wyłącznie na zbiorze treningowym, co eliminuje wyciek danych (data leakage).

5. Metodologia i Feature Engineering

Cecha	Opis
TF-IDF (nazwa dokumentu)	Wektoryzacja tekstu nazwy faktury przy użyciu TF-IDF z max_features=3000. Oddaje semantykę opisu transakcji – kluczowy sygnał dla klasyfikatora. Zapisywany jako vectorizer_nazwa_nn.pkl (Keras) i vectorizer_nazwa.pkl (XGBoost).
OneHotEncoder (typ transakcji)	Binarny wektor dla kolumny typ (BOOK/ADVANCED) – informacja o reżimie ewidencji.
OneHotEncoder (typ_pozycji)	Binarny wektor dla EXPENDITURE/INCOME – kluczowy sygnał dla kolumny KPiR.
User Embedding / LabelEncoder (company_id)	Keras: Embedding(dim=32) mapuje identyfikator firmy na gęsty wektor – model uczy się preferencji per klient. XGBoost: LabelEncoder przekształca company_id na liczbę całkowitą.

Obsługa niezbalansowanych klas: XGBoost używa compute_class_weight('balanced') z dodatkowym mnożnikiem 1.5× dla klas rzadkich. Keras stosuje EarlyStopping z monitorowaniem val_accuracy (patience=20–60) i ReduceLROnPlateau.

6. Modele uczenia maszynowego

Projekt stosuje strategię ensemble: każda predykcja wykonywana jest przez dwa niezależne modele (XGBoost i Keras). Zgodność obu daje wysoką pewność; rozbieżność sygnalizuje konieczność weryfikacji przez człowieka lub LLM.

6.1 Keras – sieci neuronowe

Blok	Opis
Input nazwy	Dense, TF-IDF (3000 cech)
Input typ	OneHot encoded
Input pozycja	OneHot encoded
Input user	Embedding(num_users, dim=32) → Flatten
Concat	Concatenate wszystkich wejść
Hidden layers	2 warstwy Dense (64–256 neuronów) z Dropout (0.1–0.5), aktywacja SELU / ReLU
Output	Dense(n_klas, activation='softmax')
Optymalizator	Adam (lr=0.001–0.003)
Loss	categorical_crossentropy
Callbacks	EarlyStopping + ModelCheckpoint + ReduceLROnPlateau

6.2 XGBoost – gradient boosting

Parametr	Wartość
max_depth	6
learning_rate	0.1
n_estimators	300
subsample	0.8
colsample_bytree	0.8
reg_alpha (L1)	1.0
reg_lambda (L2)	1.0
objective	multi:softprob / binary:logistic
sample_weight	balanced + boost ×1.5

6.3 Hyperparameter Tuning

Tuning realizowany dla modelu cel_zakupu (profil ADVANCED) przy użyciu Keras Tuner – RandomSearch / BayesianOptimization.

Metryka/Parametr	Wartość (best trial)
macro_f1	0.913

val_accuracy	98.16%
units_1 / units_2	64 / 128
dropout_1 / dropout_2	0.4 / 0.2
embedding_dim	16
learning_rate	0.003
batch_size	16
activation	SELU
batch_norm	False

7. Wyniki i metryki modeli

7.1 Wyniki XGBoost (profil KPIR)

Model	Nazwa	Accuracy	Macro F1	W-ROC AUC	M-ROC AUC
kolumna_kpir	Kolumna KPIR	85.24%	0.863	0.686	0.877
metoda_rozliczenia_podatku	Metoda rozliczenia podatku	79.87%	0.835	0.417	0.911
metoda_rozliczenia_vat	Metoda rozliczenia VAT	79.71%	0.838	0.437	0.921
odliczenie_vat	Odliczenie VAT	95.82%	0.972	0.255	0.988
cel_zakupu	Cel zakupu	93.48%	0.941	0.677	0.954
srodek_trwaly	Srodek trwały	99.82%	0.998	0.738	0.999

7.2 Wyniki Keras (profil KPIR)

Model	Nazwa	Accuracy	Obserwacje
kolumna_kpir	Kolumna KPIR	~91%	Dobra konwergencja, val_accuracy stabilna
metoda_rozliczenia_podatku	Metoda rozliczenia podatku	~80%	Klasy STD75 i STD20 wymagaja wiecej danych
metoda_rozliczenia_vat	Metoda rozliczenia VAT	~80%	VAT100 dominuje, pozostale klasy rzadkie
odliczenie_vat	Odliczenie VAT	~96%	BRAK dominuje; klasy minutarne trudne
cel_zakupu	Cel zakupu	~93%	BRAK dominuje, TAXED_SALE rozpoznawany
srodek_trwaly	Srodek trwały	~99.8%	Klasa PRAWDA rzadka; bardzo imbalanced

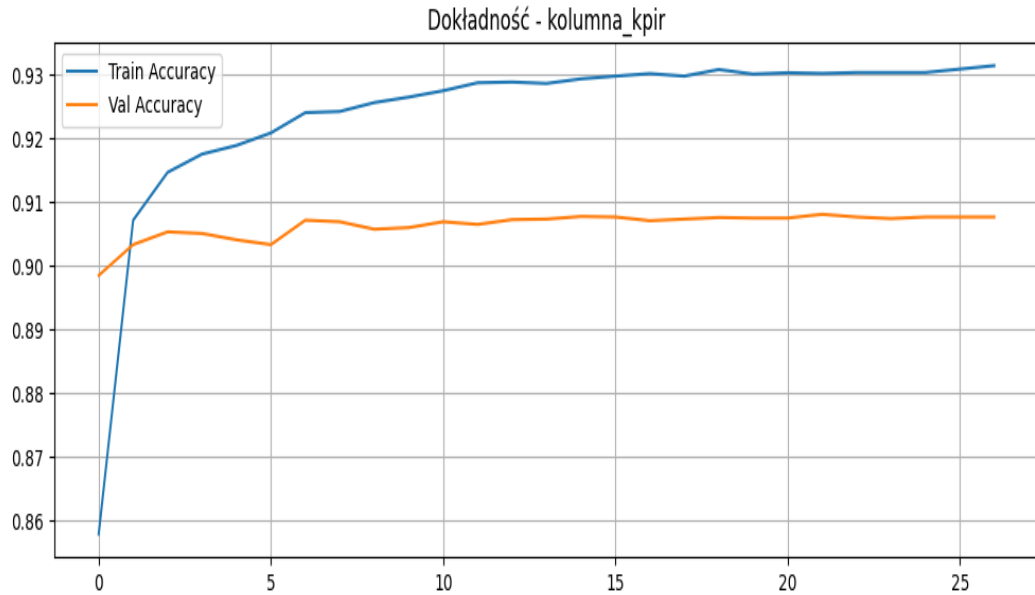
7.3 Profil ADVANCED (Ryczalt) – podsumowanie

Kolumna	Obserwacje
metoda_rozliczenia_podatku	Analogiczna do KPIR; alias metoda -> metoda
metoda_rozliczenia_vat	VAT100 dominuje; VAT50/VAT50NETTO wymagaja dowazania
odliczenie_vat	Silnie imbalanced – BRAK ~99%; klasy DEDUCTION_VAT_S_* rzadkie
cel_zakupu	BRAK dominuje; tuning poprawil macro F1 do 0.913
srodek_trwaly	PRAWDA ekstremalna rzadkosc; accuracy ~99.8%

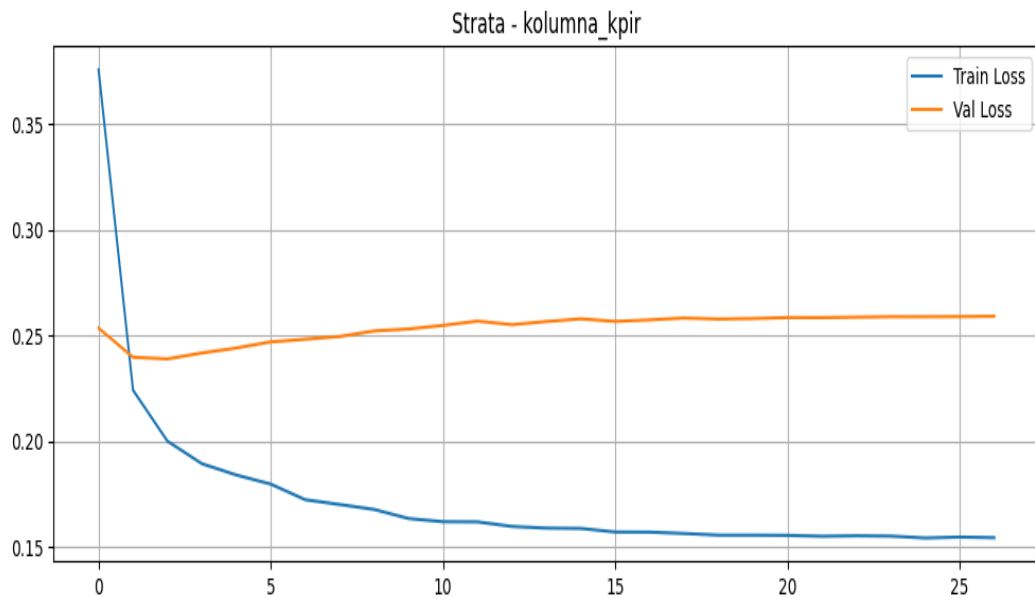
8. Wykresy z trenowania (Keras)

Wykresy accuracy i loss dla każdej z sześciu kolumn (profil KPiR).

8.1 Kolumna KPiR

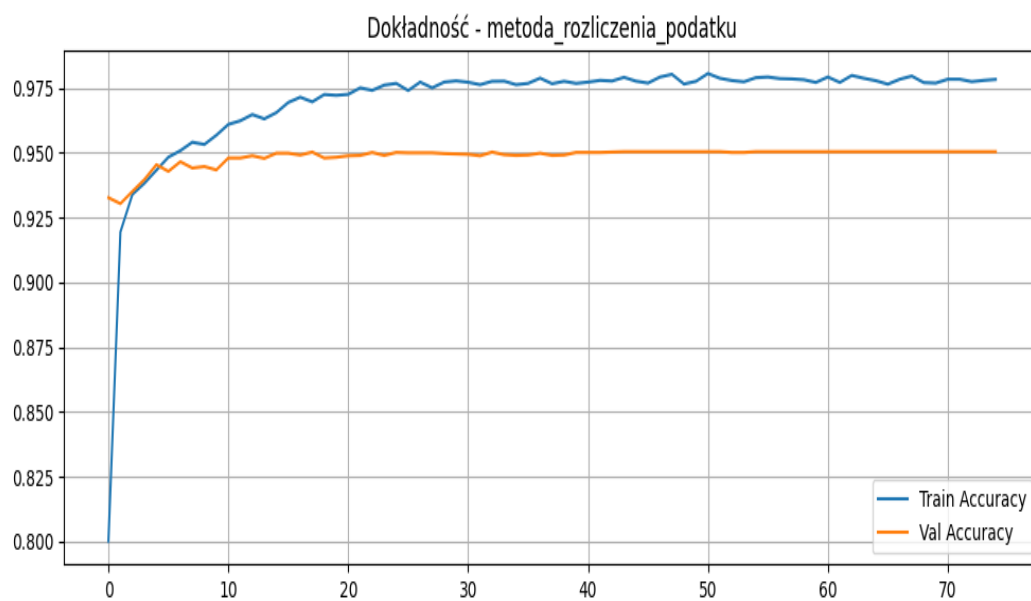


accuracy_kolumna_kpir.png

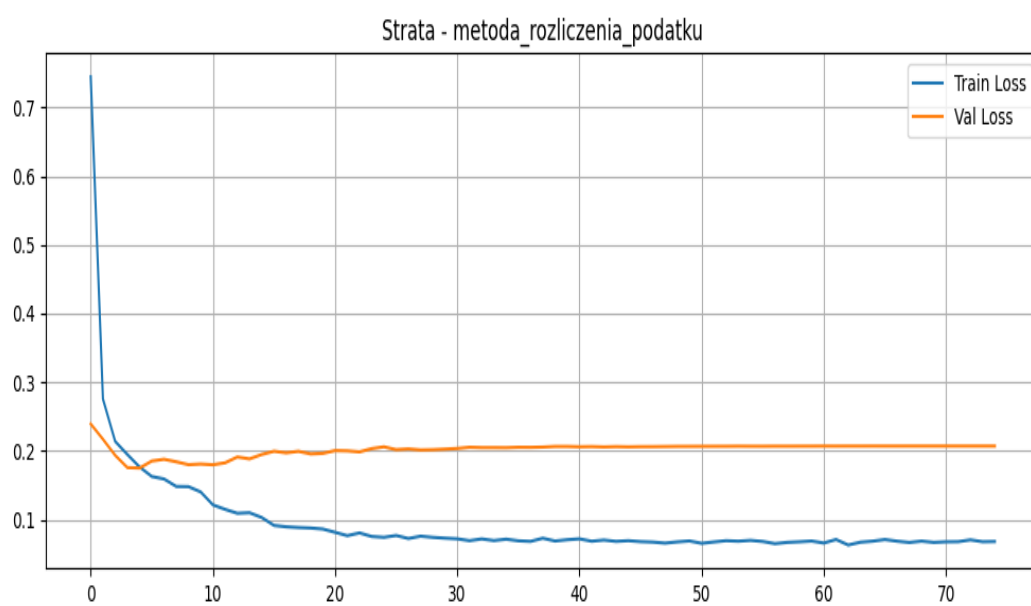


loss_kolumna_kpir.png

8.2 Metoda rozliczenia podatku

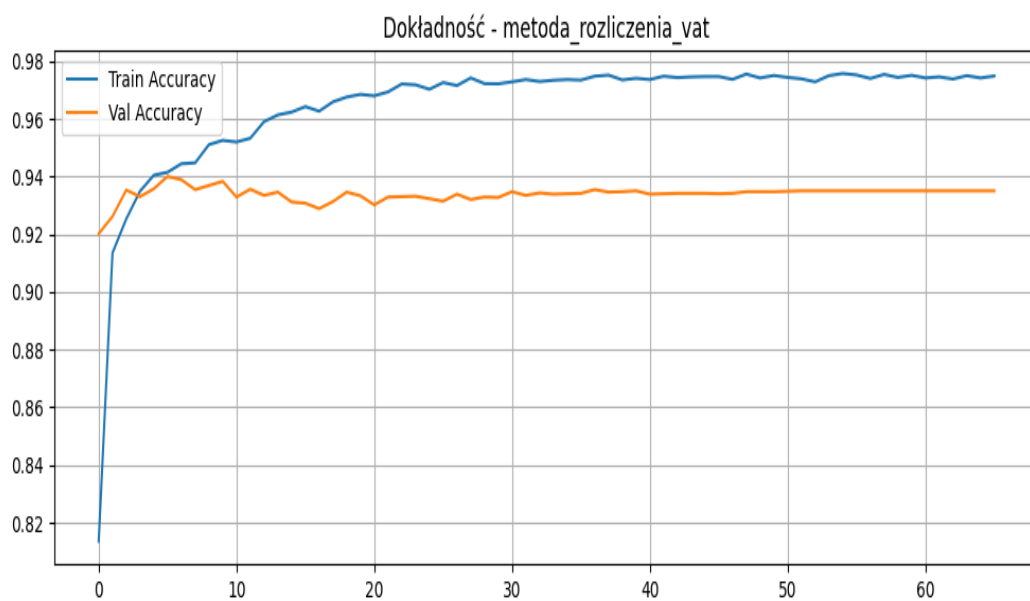


accuracy_metoda_rozliczenia_podatku.png

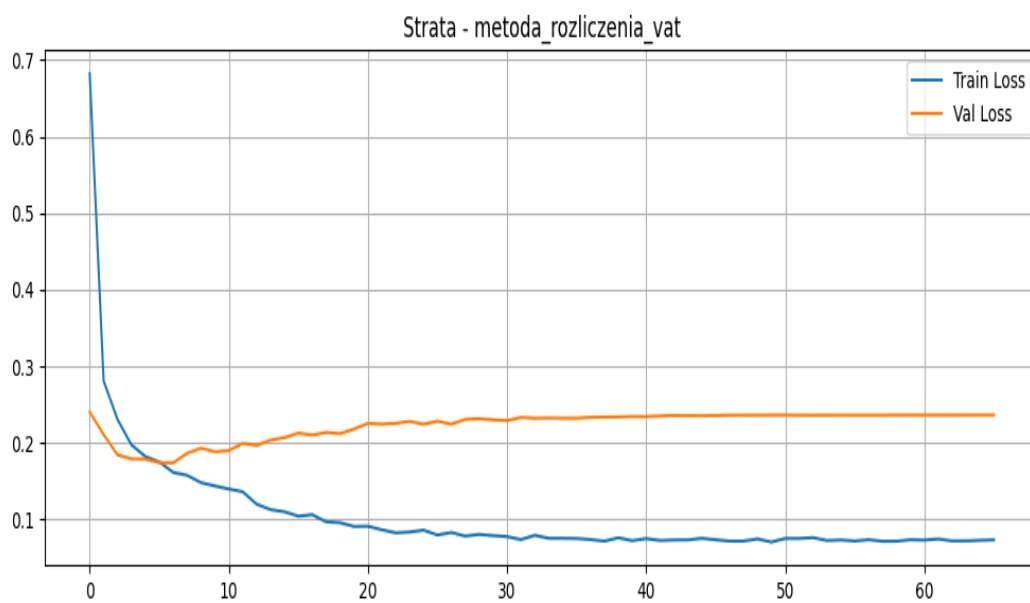


loss_metoda_rozliczenia_podatku.png

8.3 Metoda rozliczenia VAT

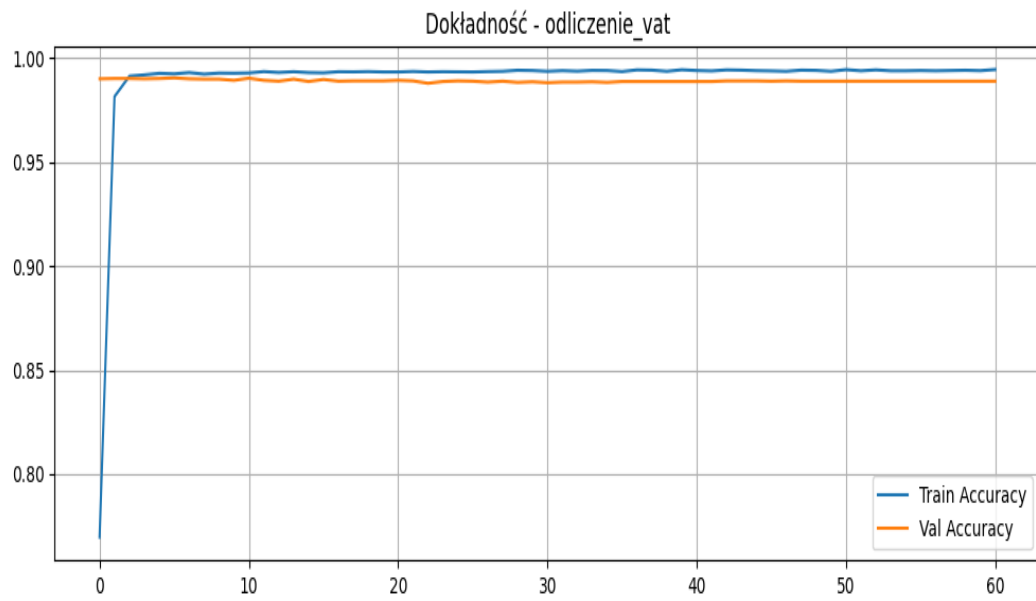


accuracy_metoda_rozliczenia_vat.png

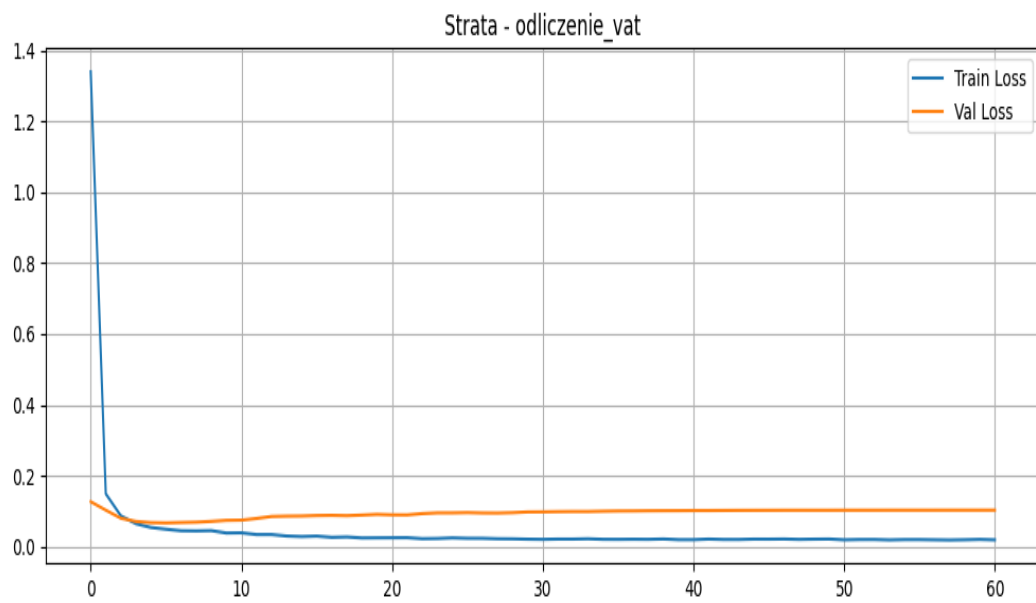


loss_metoda_rozliczenia_vat.png

8.4 Odliczenie VAT

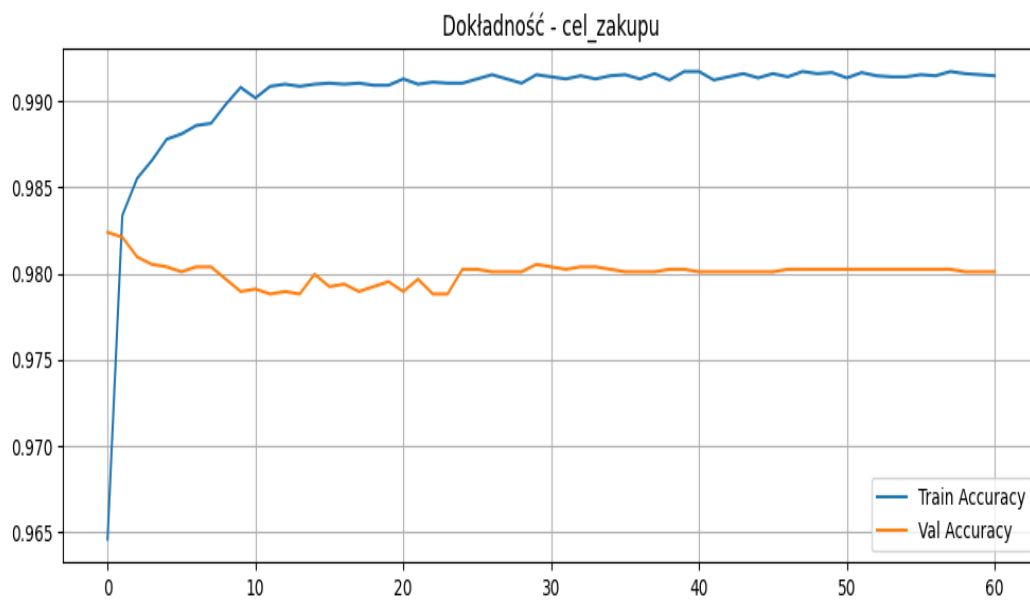


accuracy_odliczenie_vat.png

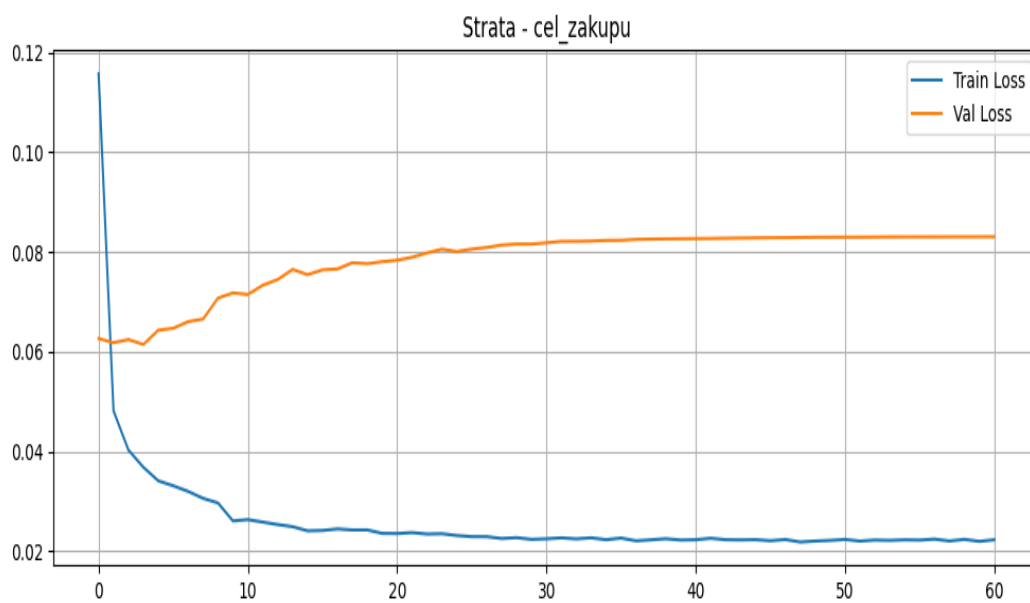


loss_odliczenie_vat.png

8.5 Cel zakupu

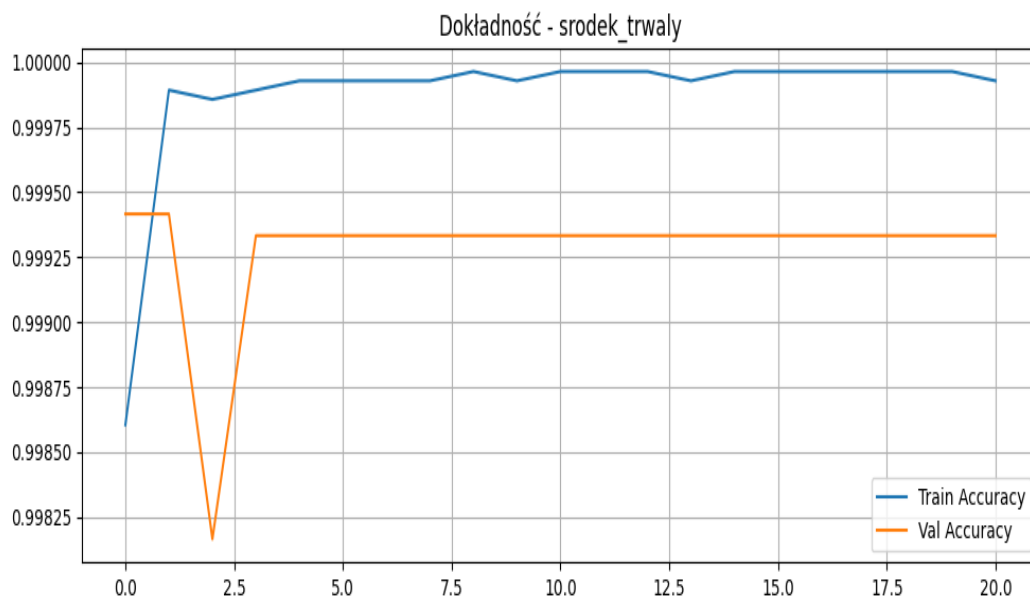


accuracy_cel_zakupu.png

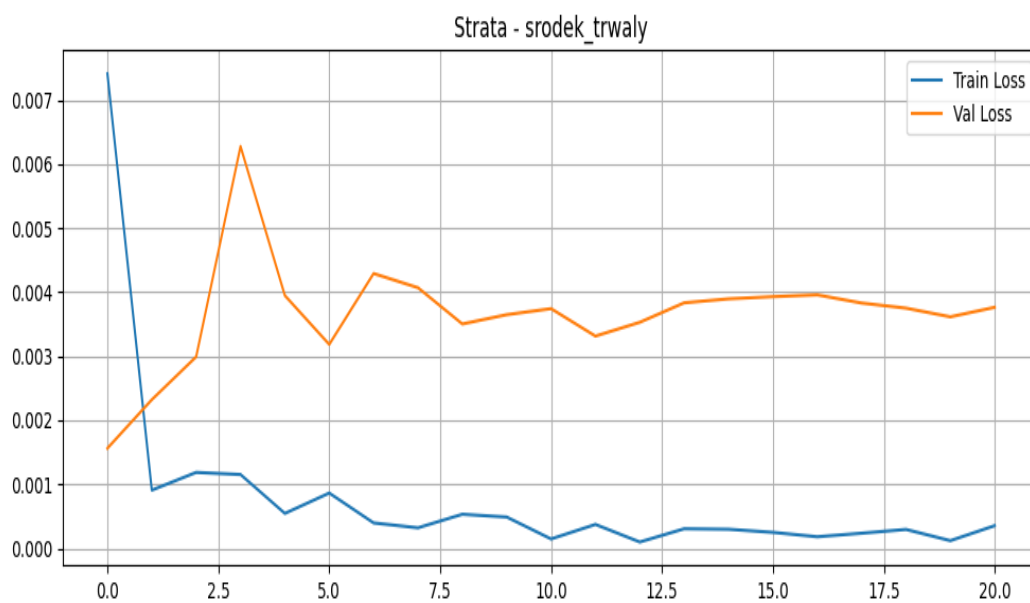


loss_cel_zakupu.png

8.6 Srodek trwaly



accuracy_srodek_trwaly.png



loss_srodek_trwaly.png

9. Architektura API (FastAPI)

9.1 Endpointy REST

Metoda	Ścieżka	Opis
POST	/predict	Predykcja kategorii; logika full-flow
POST	/predict_log	Predykcja z zapisem logu do pliku JSON
POST	/bookkeeping/confirm	Zapis potwierdzonej kategoryzacji do historii
GET	/bookkeeping/history	Przeglądanie historii (paginacja)
DELETE	/bookkeeping/history/{id}	Usunięcie wpisu z historii
PUT	/bookkeeping/company-config	Ustawienie progów i flagi LLM per firma
GET	/bookkeeping/company-config/{id}	Pobranie konfiguracji firmy
GET	/bookkeeping/llm-usage/{id}	Użycie LLM per firma (tokeny, koszty)
GET	/bookkeeping/llm-usage-report	Raport zbiorczy użycia LLM
GET	/bookkeeping/llm-usage-clients	Lista firm z użyciem LLM
GET	/health	Healthcheck (status: ok)

9.2 Pola żądania /predict

Pole	Typ	Opis
nazwa	str	Nazwa dokumentu/faktury – główna cecha modelu
typ_pozycji	str	EXPENDITURE lub INCOME
company_id	str int float	Identyfikator firmy (personalizacja)
accounting_type	str	kpir lub advance

9.3 Fallback LLM (AWS Bedrock)

Gdy $\text{similarity_historii} < \text{confidence_ai_threshold}$ i firma ma $\text{llm_enabled}=\text{True}$, system wywołuje AWS Bedrock Converse API. Użycie tokenów rejestrowane w llm_usage .

9.4 Historia SQLite + FTS5

FTS5 full-text search + cosine similarity. $\text{similarity} \geq \text{confidence_exact} \rightarrow \text{historical_match}$ bez uruchamiania modeli ML.

10. Infrastruktura i wdrożenie

Komponent	Szczegóły
Docker	scripts/run_docker.sh: build + run :8000 + health check /health
AWS EC2/ECS	Produkcja: .env.aws.example, uvicorn 0.0.0.0:8000
SQS Consumer	scripts/run_sqs_consumer.sh – cron lub ciagly
ENV (klucz)	BOOKED_DB_PATH, DEFAULT_CONFIDENCE_*, LLM_BEDROCK_MODEL_ID, SQS_QUEUE_URL

10.1 Zmienne środowiskowe

Zmienna	Opis
BOOKED_DB_PATH	Ścieżka do pliku SQLite historii
DEFAULT_CONFIDENCE_EXACT	Próg historii (domyślnie 0.90)
DEFAULT_CONFIDENCE_AI	Próg modeli ML (domyślnie 0.70)
LLM_BEDROCK_MODEL_ID	ID modelu AWS Bedrock
SQS_QUEUE_URL	URL kolejki SQS
AWS_ACCESS_KEY_ID	Klucz AWS (env / IAM role)

11. Testy integracyjne

Plik testu	Zakres
test_api_integration.py	Smoke-testy API: /health, walidacja wejsc, predict dla nowej firmy, confirm + historical_match
test_db_llm_usage.py	Testy warstwy DB: rejestracja uzycia LLM, agregacja monthly, raporty, upsert konfiguracji
test_llm_client.py	Testy klienta LLM: is_enabled() z roznymi zmiennymi env, request_bedrock gdy disabled
test_llm_contracts.py	Kontrakty LLM: struktura payload escalation, build_final_prediction consensus/rozbiez.
test_similarity_performance.py	Testy wydajnosciowe wyszukiwania podobienstwa: czas odpowiedzi, poprawnosc dopasowania
test_sqs_consumer.py	Testy consumera SQS: przetwarzanie wiadomosci, obsluga bledow

Uruchomienie: pytest tests/ -v

12. Przypadki testowe (UAT – wyniki rzeczywiste)

Rzeczywiste wyniki testów przeprowadzonych na działającym systemie. Źródło: Model Consensus decision.txt (maj 2026).

ID	Scenariusz	Wejscie (nazwa)	Oczekiwana decyzja	Faktyczna decyzja	Wynik predykcji	Status	Uwagi
TC-001	Faktura telefoniczna – historical_match po confirm	Abonament tel. Orange 05/2026 / firma-123 / KPIR	historical_match	historical_match	OTHER_EXPENSES / STD100 / STD23 / VAT100 / FIRM / NIE (sim=1.0)	PASS	Pelne dopasowanie FTS5 po wcześniejszym confirm
TC-002	Faktura telefoniczna – model_consensus (pierwsze zapytanie)	Abonament tel. Orange 05/2026 / firma-123 / KPIR	model_consensus	model_consensus	OTHER_EXPENSES / STD100 / VAT100 / BRAK / BRAK / FALSE; XGB=Keras	PASS	sim=0.8685 ponizej confidence_exact=0.92
TC-003	Wynagrodzenie pracownika – LLM fallback (llm_enabled=True)	Lista plac kwiecien 2026 / firma-123 / KPIR	llm_fallback_used	llm_fallback_used	OTHER_EXPENSES / STD100 / STD23 / VAT100 / FIRM / NIE (conf=0.78)	PASS	qwen3-32b-v1:0; tokeny: in=1079 out=113 tot=1192; best_sim=0.19
TC-004	Wynagrodzenie pracownika – ai_assist (llm_enabled=False)	Lista plac kwiecien 2026 / firma-123 / KPIR	ai_assist_required	ai_assist_required	XGB+Keras: OTHER_EXPENSES/STD100 /VAT100; top_similar z historii	PASS	Brak LLM; best_sim=0.19; kieruje do recznej weryfikacji
TC-005	Zakup srodka trwalego – model_consensus	Dell XPS 15 / laptop sluzbowy / firma-123 / KPIR	model_consensus / srodek_trwaly=TRUE	model_consensus	OTHER_EXPENSES / STD100 / VAT100 / BRAK / BRAK / TRUE	PASS	XGB i Keras zgodne na srodek_trwaly=TRUE; czesciowa rozbiez. vat
TC-006	Nowy klient – brak historii, rozbiez. kolumna_kpir	Pierwsza faktura TEST_CO_001 / new-company-001 / KPIR	ai_assist_required	ai_assist_required	XGB: OTHER_EXPENSES; Keras: PURCHASE_OF_TRADING_GOODS	PASS	best_sim=0.0; brak historii; rozbiez. na kolumna_kpir
TC-007	LLM fallback – szczegoly wywolania AWS Bedrock	Lista plac kwiecien 2026 / firma-123 / KPIR (LLM=ON)	llm_fallback_used	llm_fallback_used	model=qwen.qwen3-32b-v1:0; in=1079 /out=113/total=1192 tokenow	PASS	region=eu-central-1; confidence=0.78; AWS Bedrock Converse API
TC-008	Confirm + ponowna predykcja = historical_match	Abonament tel. Orange 05/2026 / firma-123 / KPIR	historical_match (sim=1.0)	historical_match	OTHER_EXPENSES / STD100 / STD23 / VAT100 / FIRM / NIE (sim=1.0)	PASS	Po /bookkeeping/confirm; FTS5 zwraca identyczne dopasowanie
TC-009	Profil ADVANCED (ryczalt) – rozbiez. srodek_trwaly, LLM=OFF	Abonament hosting AWS 05/2026 / firma-321 / advance	ai_assist_required	ai_assist_required	XGB: srodek=FALSE; Keras: srodek=PRAWDA; best_sim=0.62	PASS	accounting_type=advance; sim ponizej confidence_exact=0.92

TC-010	Walidacja – brak wymaganych pol wejsciowych	(brak: nazwa / typ_pozycji / company_id / accounting_type)	HTTP 400	HTTP 400	error: Brak wymaganych pol: nazwa, typ_pozycji, company_id, accounting_type	PASS	Poprawna walidacja wejścia API
--------	---	--	----------	----------	---	------	--------------------------------

12.1 Przykładowe odpowiedzi API

Rzeczywiste odpowiedzi systemu dla kluczowych scenariuszy testowych:

Model Consensus – Abonament tel. Orange 05/2026 (firma-123)

Decyzja: decision: "model_consensus"

```
kolumna_kpir: OTHER_EXPENSES
metoda_rozliczenia_podatku: STD100
metoda_rozliczenia_vat: VAT100
odliczenie_vat: BRAK
cel_zakupu: BRAK
srodek_trwaly: FALSE
matched_invoice.similarity: 0.8685 (ponizej confidence_exact=0.92)
```

LLM Fallback – Lista plac kwiecien 2026 (llm_enabled=True)

Decyzja: decision: "llm_fallback_used"

```
provider: aws-bedrock | region: eu-central-1
model: qwen.qwen3-32b-v1:0 | confidence: 0.78
kolumna_kpir: OTHER_EXPENSES
metoda_rozliczenia_podatku: STD100
metoda_rozliczenia_vat: STD23
odliczenie_vat: VAT100 | cel_zakupu: FIRM | srodek_trwaly: NIE
tokeny: input=1079 / output=113 / total=1192
```

Historical Match – Abonament tel. Orange (po confirm)

Decyzja: decision: "historical_match"

```
kolumna_kpir: OTHER_EXPENSES
metoda_rozliczenia_podatku: STD100
metoda_rozliczenia_vat: STD23
odliczenie_vat: VAT100 | cel_zakupu: FIRM | srodek_trwaly: NIE
matched_invoice.similarity: 1.0 (identyczne dopasowanie)
```

Nowy klient – rozbiez. modeli (new-company-001)

Decyzja: decision: "ai_assist_required"

```
XGBoost: kolumna_kpir=OTHER_EXPENSES
Keras: kolumna_kpir=PURCHASE_OF_TRADING_GOODS
best_similarity: 0.0 (brak historii)
Kieruje do recznej weryfikacji lub LLM
```

Profil ADVANCED – Abonament hosting AWS (firma-321)

Decyzja: decision: "ai_assist_required" (accounting_type=advance)

XGBoost: srodek_trwaly=FALSE

Keras: srodek_trwaly=PRAWDA

best_similarity: 0.6233 (ponizej confidence_exact=0.92)

ai_hint_request zwrócony do dalszego przetwarzania

Walidacja – brak pol wejsciowych

Decyzja: HTTP 400

error: "Brak wymaganych pol: nazwa, typ_pozycji, company_id, accounting_type"

13. Wnioski końcowe

Skuteczność modeli ML

Modele osiągają wysoką dokładność dla klas dominujących (srodek_trwały ~99.8%, cel_zakupu ~93.5%, kolumna_kpir ~85-91%). Klasy rzadkie wymagają dalszego zbierania danych lub technik oversampling (SMOTE), co jest naturalnym ograniczeniem przy imbalanced datasets.

Strategia ensemble

Połączenie XGBoost i Keras w architekturze dual-model podwyższa niezawodność predykcji. Konsensus modeli jest silnym sygnałem jakości decyzji; rozbież. automatycznie eskaluje do człowieka lub LLM.

Personalizacja per firma

Mechanizm User Embedding (Keras) i LabelEncoder (XGBoost) w połączeniu z historią SQLite per firma tworzy efektywny system personalizacji. Im dłużej firma korzysta z systemu, tym więcej dopasowań pochodzi z historii (historical_match), redukując obciążenie modeli ML.

Skalowalność architektury

Architektura REST API + Docker + SQS umożliwia skalowanie synchroniczne (HTTP) jak i asynchroniczne (kolejka). Separacja treningu od serwowania pozwala na niezależne aktualizacje modeli bez przerw w działaniu API.

LLM jako siatka bezpieczeństwa

AWS Bedrock stanowi ostatnią linię obrony dla przypadków granicznych. System rejestruje tokeny i koszty per firma, co umożliwia kontrolę kosztów i selektywne włączanie LLM dla wybranych klientów.

Gotowość produkcyjna

Docker, FastAPI, testy integracyjne, healthcheck, runbooki AWS – projekt spełnia wymagania produkcyjne klasy enterprise.

14. Rekomendacje i dalszy rozwój

- Rozbudowa zbioru danych dla klas rzadkich – SMOTE / aktywne uczenie
- Embeddingi semantyczne (BERT / sentence-transformers) zamiast TF-IDF
- Automatyczny feedback loop – re-trening po potwierdzeniach /bookkeeping/confirm
- Monitoring driftu danych (Evidently AI) + alerty na degradację accuracy
- Multi-tenant fine-tuning – oddzielne modele dla dużych firm
- Rozszerzenie profili: pełna księgowość (KH), inne reżimy podatkowe

Projekt Kategorizator stanowi dojrzałe, produkcyjne rozwiązanie klasy enterprise dla automatyzacji procesów księgowych w ekosystemie Symfonia.